

SignAI Backend (FastAPI & VideoMAE)

This backend serves as the deep learning inference engine for the SignAI real-time web application. It processes raw webcam frame sequences from the frontend and uses a highly optimized Vision Transformer (VideoMAE) to classify Indian Sign Language (ISL) sentences with over 74% multi-class accuracy.

System Architecture

The SignAI pipeline operates via a real-time HTTP loop:

1. **Frontend Capture:** The React/Vite frontend accesses the user's webcam and captures frames at a normalized framerate.
2. **Buffer Aggregation:** The frontend groups these frames into sliding windows (buffers) and batch-encodes them as base64 strings.
3. **API Transmission:** The batch is sent as a JSON payload to the FastAPI `/predict-sentence` endpoint.
4. **Backend Preprocessing:** The backend decodes the base64 frames back into RGB images. It strictly trims/pads the buffer to exactly **16 frames**, crops them to `224x224` pixels, and normalizes the tensor using standard ImageNet mean/standard deviation.
5. **Model Inference (VideoMAE):** The 16-frame 4D tensor `(1, 16, 3, 224, 224)` is passed into the Video Masked Autoencoder (VideoMAE).
6. **Classification Head:** The custom classification head outputs logit probabilities for 101 distinct ISL sentence classes.
7. **Response:** The highest probability class (if above the threshold) is converted to human-readable text and returned to the frontend along with latency metrics.

Model & Training Details

Core Algorithm: VideoMAE (Video Masked Autoencoder)

Instead of using older sequential models like CNN+LSTM or CNN+RNN, this project utilizes **Hugging Face's VideoMAEForVideoClassification**.

- **Why VideoMAE?:** VideoMAE uses a purely Vision Transformer (ViT) architecture. Instead of processing frame 1, then frame 2 sequentially, it stacks all 16 frames into a "3D Spacetime Cube". It then uses Self-Attention to mathematically compare the relationship between *every patch in every frame simultaneously*, naturally capturing the fluid motion of human hands and arms across time.
- **Pre-trained Backbone:** Initialized using `Shawon16/VideoMAE_WLASL_250_epochs` (trained on the American Sign Language dataset) to leverage its baseline understanding of human hand gestures.

Dataset

- **Target Classes:** 101 custom Indian Sign Language (ISL) sentence/phrase gestures (e.g., "I am feeling cold", "Bring water for me").
- **Volume:** 144,448 cleanly extracted frames split across 7,671 distinct sequences (`data_sentence_clean/`).

- **Format Requirements:** Every sequence fed to the model must be exactly **16 frames** long at **224x224 RGB**.

Training Pipeline (`train_videomae.py`)

- **Optimizer:** AdamW with weight decay (`0.05`) to prevent overfitting.
- **Loss Function:** `CrossEntropyLoss` with **Inverse Class Weighting**. Because the dataset is highly imbalanced (some signs have 100 videos, others have 500), inverse weighting scales up the penalty for getting rare signs wrong, forcing the model to learn all 101 classes equally.
- **Differential Learning Rates:** The ViT backbone is fine-tuned at a very low learning rate (`2e-5`) to preserve its base knowledge, while the new 101-class output head learns at a higher rate (`1e-4`).
- **Validation Performance:** Achieved **~74% Top-1 Accuracy** across 101 complex dynamic classes by Epoch 10, with early stopping implemented to prevent validation loss divergence.

Setup & Execution

1. Environment Preparation

Ensure you are using the correct Python virtual environment (`.venv`) located at the root of the workspace.

```
# Windows PowerShell
..\venv\Scripts\Activate.ps1
```

2. Install Dependencies

```
cd backend
pip install -r requirements.txt
```

3. Start the Server

Run the FastAPI web server using Uvicorn. It runs on `localhost:8000` by default.

```
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

API Endpoints

`GET /health`

Returns service status, GPU availability, and whether the model weights successfully loaded into VRAM.

`POST /predict-sentence` (Main Real-Time Route)

Accepts a batch of exactly 16 sequential base64 image frames.

Request Body:

```
{
  "frames": ["<base64-str-frame-1>", "<base64-str-frame-2>", "... up to 16"]
}
```

Response:

```
{
  "sentence": "i am feeling cold",
  "confidence": 0.88,
  "frame_count": 16,
  "latency_ms": 112
}
```

POST /predict (Legacy/Fallback)

Retained for backwards compatibility with single-frame (static) gesture testing if required by older frontend components.

 Backend Structure

- `/app/main.py`: FastAPI server configuration and routing.
- `/app/services/sentence_service.py`: Contains the `VideoMAE` initialization logic, base64 decoding buffer, and tensor transformations.
- `/checkpoints/`: Holds exactly two critical files:
 - `best_videomae_model.pt` (The 1GB dynamically trained model weights).
 - `videomae_classes.json` (The exact ID-to-String mapping for the 101 classes).
- `/data_sentence_clean/`: The finalized frame-dataset used for training.
- `/scripts/`: Data isolation/preparation and metric-generation scripts.
- `/training/`: The PyTorch loop logic that generated the `best_videomae_model.pt` file.